

Правительство Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
(НИУ ВШЭ)

Московский институт электроники и математики им. А.Н. Тихонова

Проектная документация по проекту №1963
«Мастерская YADRO: Разработка лабораторного практикума по верификации RISC-V»

Участники проекта гр. БИБ221

И. Ю. Власов
А. А. Добротин
В. С. Доманова
М. И. Карамнов
А. А. Тарало
В. Р. Тен

Руководители
Я. Е. Мещеряков
А. Ю. Романов

Москва, 2025

СОДЕРЖАНИЕ

Разработанный постер и дизайн-код проекта	3
Система сборки документации	5
Модуль biriscv_frontend	6
Параметры	6
Входы и выходы	6
Входные сигналы	6
Выходные сигналы	6
Подключенные модули	6
Модуль biriscv_npc	7
Основные функции	7
Модуль biriscv_decode	8
Основные функции	8
Модуль biriscv_fetch	8
Модуль dcache	10
Параметры	10
Входы и выходы	10
Входные сигналы процессорного интерфейса	10
Входные сигналы AXI интерфейса	10
Выходные сигналы процессорного интерфейса	10
Выходные сигналы AXI интерфейса	11
Подключенные модули	11
Модуль dcache_if_rmem	11
Назначение	11
Ключевые особенности	11
Основные функции	11
Модуль dcache_rmem_mux	12
Назначение	12
Ключевые особенности	12
Основные функции	12
Модуль dcache_mux	12
Назначение	12
Ключевые особенности	12
Основные функции	13
Модуль dcache_axi	13
Назначение	13
Ключевые особенности	13
Основные функции	13
Модуль dcache_core	14
Назначение	14
Ключевые особенности	14
Основные функции	14
Особенности реализации	14
Модуль riscv_core	15

Параметры	15
Входы и выходы	15
Подключенные модули	15
Модуль biriscv_frontend	15
Модуль biriscv_mmu	17
Модуль biriscv_lsu	17
Модуль biriscv_csr	18
Модуль biriscv_multiplier	19
Модуль biriscv_divider	19
Модуль biriscv_issue	20
Модуль biriscv_exec	21
Модуль riscv_tcm_top	22
Параметры	23
Входы и выходы	24
Системные сигналы	24
Подключенные модули	25
1. riscv_core	25
Модуль dport_mux	25
Назначение	25
Ключевые особенности	25
Основные функции	25
Модуль tcm_mem	25
Назначение	26
Ключевые особенности	26
Основные функции	26
Модуль dport_axi	26
Назначение	26
Ключевые особенности	26
Основные функции	27
Модуль riscv_top	28
Параметры	28
Порты	28
Подключенные модули	28
1. riscv_core	28
2. icache	30
3. dcache	30
Виртуальная плата DE2-115	32
Описание проекта	32
Основные возможности	32
Требования	32
Для графического клиента	32
Для сервера	32
Установка	33
Использование	33
Запуск через Makefile	33

Запуск графического клиента	33
Файл конфигурации	33
Запуск сервера	33
Функционал графического интерфейса	33
Makefile	34
Таблица сигналов	34
Виртуальная плата DE10-Lite	35
Описание проекта	35
Основные возможности	35
Требования	35
Для графического клиента	35
Для Verilog сервера	35
Установка	36
Использование	36
Запуск через Makefile	36
Запуск графического клиента	36
Файл конфигурации	36
Запуск сервера	36
Функционал графического интерфейса	36
Makefile	37
Таблица сигналов	37

Разработанный постер и дизайн-код проекта

Постер – одна из важнейших частей проекта. Благодаря нему, на постерной сессии можно продемонстрировать себя лучшим образом. Постер должен быть информативен, прост к чтению и выглядит красиво, привлекая внимание

В качестве редактора было принято решение использовать онлайн-редактор Figma

В рамках разработки постера, изначально был создан макет, похожий по стилю на айдентику ГЭС-2. Основной задачей было показать примерную структуру расположения блоков текста и показать как что можно расположить

Затем, было разработано две версии – строгая и экспериментальная, которые соответствовали требованиям команды. Намерено использовалось высокое расширение, дабы постер выглядел органично на большом формате A1

Экспериментальная версия опирается на макет с ГЭС-2 но привносит в него цвета и роадамап в виде диаграммы Гаанта. Однако, к сожалению, она оказалась гораздо менее читаемой.

Более лаконичная версия постера, опирается на айдентику НИУ ВШЭ, что делает его куда более вписывающимся в общую канву постерной сессии. Роадамап представлен в виде таймлайна с точками для удобной визуализации. Он располагается посередине, благодаря чему потенциальному эксперту бросается в глаза сразу же

#1963

Разработка лабораторного практикума по верификации RISC-V

Цель работы

Разработать лабораторный практикум по методам верификации процессорных ядер RISC-V и их периферии

Команда

- Ярослав Мещеряков
руководитель проекта
- Александр Романов
руководитель направления
- Методисты
Иван В., Сергей Д., Алексей К.
- Системные программисты
Максим Г., Владимир Т.
- Верификаторы
Арсений Д., Антон Т.

Технологии

SystemVerilog Verilog C++

RISC-V AMBA Make

Требования и планируемые результаты

На выходе с проекта будут получены следующие материалы:

Лабораторная работа #1
Исследование, верификация АЛУ

Методический материал по изучению работы арифметико-логического устройства и реализации его на языке SystemVerilog, его верификация

Лабораторная работа #2
Системные шины: AXI, APB

Методический материал по изучению понятия системной шины, а также конкретно системных шин AXI и APB. Верификация подключенной по системной шине к ядру RISC-V периферии

Текущие результаты

Материалы для работы #1: АЛУ

Создано методическое пособие по внутреннему устройству и верификации АЛУ. Разработаны материалы для ЛР в виде RTL-Дизайна (верифицируемое устройство) и верификационного окружения с помощью технологий SystemVerilog, Verilog, C++.

Материалы для работы #2: Шины

Разработаны материалы для ЛР:

- RTL-Дизайн - периферия с интерфейсами APB, мост AXI<-> APB для соединения с ядром RISC-V
- Верификационное окружение с помощью технологий SystemVerilog, Verilog, C++.
- ПО для ядра RISC-V, позволяющее взаимодействовать с периферией (код на C)

Дорожная карта



Лабораторная работа #2



Проблемы

Разнообразие версий хост-систем

Системы, на которых запускается ПО для верификации, имеют разные версии ПО по умолчанию, что приводит к сложности автоматизации и дублированию записей на разных системах.

Решение: проанализировать все зависимости используемого ПО, составить список протестированных периодов, начать выполнение лабораторных работ.

В перспективе планируется перейти на развертывание работ в Docker



Репозиторий с кодом

ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ

Система сборки документации

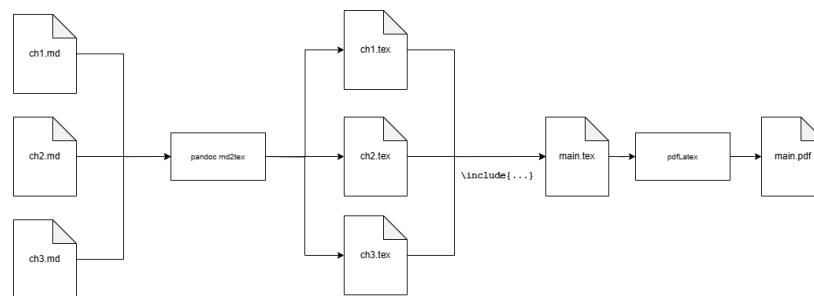
Достаточно большой болью является компоновка файлов документации из нескольких источников. Обычно, это делается благодаря кооперативному написанию текста в редакторах вроде Google Docs, Office 365 и их аналогам (например, российские Р7 офис или Яндекс на базе OnlyOffice).

Однако, более правильным подходом в командах, связанных с программированием, считается использование парадигмы Documentation as a Code (DaaC). Вся документация храниться в отдельном репозитории. Преимущества такого подхода очевидны

- Использование привычных инструментов для написания кода для документации
- Полноценное версирование благодаря использованию Git. Используя знакомый всем разработчикам CLI интерфейс спокойно
- Возможность удобного ревью и вливания написанных изменений в общий репозиторий

Дополнительным плюсом является возможность использования CI систем, предоставляемых современными SourceControl решениями.

Объединив все эти требования, родилась идея разработать несложную систему для сборки документации в уже готовые PDF файлы



Принцип работы сборки документации

Сборка происходит в два этапа

1. Исходные файлы, с использованием pandoc конвертируются в tex файлы. Все изображения и подобные лежат в папке imgs
2. Благодаря заранее указанным директивам include в main.tex они импортируются и собираются в единый документа

Так как основной системой в проекте является GitLab Ci, то был также разработан специальный конфиг для вызова скрипта.

Из-за того, что latex достаточно требователен к среде исполнения, а потому стоит использовать Docker образ для сборки

Из важных особенностей, также используется несколько lua-скриптов для сборщика, который - заменяет код для изображений, устанавливая размеры и помещая в корректное окружение его - изменяет код для таблиц, делая их отображение корректным

Модуль biriscv_frontend

Модуль представляет собой фронтенд 32-битного RISC-V процессора, отвечающий за: - Предсказание переходов - Выборку инструкций из кэша - Предварительное декодирование инструкций - Формирование конвейера инструкций для двух потоков выполнения

Параметры

Параметр	Описание
SUPPORT_BRANCH_PREDICTION	Включение предсказания переходов
SUPPORT_MULDIV	Поддержка инструкций умножения/деления
SUPPORT_MMU	Поддержка MMU
EXTRA_DECODE_STAGE	Добавление дополнительной стадии декодирования
NUM_BTBT_ENTRIES	Число записей в Branch Target Buffer
NUM_BTBT_ENTRIES_W	Ширина адреса BTB
NUM_BHT_ENTRIES	Число записей в Branch History Table
NUM_BHT_ENTRIES_W	Ширина адреса BHT
RAS_ENABLE	Включение стека возвратов
GSHARE_ENABLE	Включение схемы предсказания Gshare
BHT_ENABLE	Включение истории переходов
NUM_RAS_ENTRIES	Глубина стека возвратов
NUM_RAS_ENTRIES_W	Ширина адреса RAS

Входы и выходы

Входные сигналы

Сигнал	Описание
clk_i	Тактовый сигнал
rst_i	Сигнал сброса
icache_accept_i	Подтверждение приема запроса кэшем инструкций
icache_valid_i	Действительные данные из кэша инструкций
icache_error_i	Ошибка доступа к кэшу инструкций
icache_inst_i[63:0]	Считанные инструкции (2x32-bit)
icache_page_fault_i	Ошибка страницы MMU
fetch0_accept_i	Подтверждение приема 0-го потока
fetch1_accept_i	Подтверждение приема 1-го потока
fetch_invalidate_i	Сигнал инвалидации конвейера
branch_request_i	Запрос на обработку перехода
branch_pc_i[31:0]	Адрес перехода
branch_priv_i[1:0]	Привилегированный режим
branch_info_*	Информация о выполненных переходах

Выходные сигналы

Подключенные модули

В данном модуле подключено три сабмодуля:

Сигнал	Описание
icache_rd_o	Сигнал чтения кэша инструкций
icache_flush_o	Сброс кэша инструкций
icache_invalidate_o	Инвалидация кэша инструкций
icache_pc_o[31:0]	Адрес запрашиваемой инструкции
icache_priv_o[1:0]	Привилегированный режим для запроса
fetch0_*_o	Выходные сигналы для 0-го потока (инструкция, РС, флаги)
fetch1_*_o	Выходные сигналы для 1-го потока (инструкция, РС, флаги)

Модуль biriscv_prs

Исходный код модуля представлен в biriscv_prs.v

Назначение Модуль управления потоком команд (Next PC) в процессоре RISC-V, отвечающий за: - Определение адреса следующей исполняемой команды - Реализацию механизмов предсказания ветвлений - Оптимизацию работы конвейера за счет уменьшения потерь на ветвлениях

Ключевые особенности

1. Гибкая архитектура:
 - Настраиваемые размеры БТВ (32 записи), ВНТ (512 записей) и RAS (8 уровней)
 - Поддержка различных алгоритмов предсказания
 - Возможность полного отключения предсказания
2. Компоненты предсказания:
 - Branch Target Buffer для хранения целей переходов
 - Return Address Stack для обработки call/ret
 - Branch History Table для статистического предсказания
3. Эффективность:
 - Минимизация penalty при ошибочном предсказании
 - Поддержка спекулятивного исполнения
 - Адаптивные алгоритмы замены записей

Основные функции

1. Предсказание адресов:
 - Расчет следующего РС для последовательных команд
 - Определение целей условных/безусловных переходов
 - Обработка call/ret инструкций через RAS
2. Обучение предсказателя:
 - Обновление БТВ при обнаружении новых ветвлений
 - Корректировка ВНТ на основе фактических переходов
 - Управление состоянием RAS
3. Интеграция с конвейером:
 - Генерация сигналов принятия/отклонения ветвлений
 - Обработка недействительных предсказаний
 - Синхронизация с fetch-стадией

Модуль `biriscv_decode`

Исходный код модуля представлен в `biriscv_decode.v`

Назначение Блок декодирования инструкций в процессоре RISC-V, выполняющий: - Разбор и декодирование поступающих инструкций - Классификацию инструкций по типам выполнения - Формирование управляющих сигналов для конвейера

Ключевые особенности

1. Гибкая архитектура:
 - Поддержка двух вариантов задержки (1 или 2 такта)
 - Конфигурируемая поддержка умножения/деления
 - Обработка 32-битных и парных инструкций
2. Компоненты обработки:
 - Декодеры инструкций для каждого слота
 - Буфер FIFO для хранения и синхронизации
 - Обработка исключений и ошибок
3. Эффективность:
 - Поддержка dual-issue (2 инструкции за такт)
 - Минимизация задержек декодирования
 - Интеграция с системой предсказания ветвлений

Основные функции

1. Декодирование инструкций:
 - Определение типа операции (ALU, LSU, Branch и др.)
 - Выявление специальных инструкций (CSR, MUL/DIV)
 - Проверка валидности инструкций
2. Управление потоком:
 - Обработка прерываний и исключений
 - Реакция на неправильные предсказания ветвлений
 - Синхронизация с fetch-стадией
3. Распределение по исполнительным блокам:
 - Формирование сигналов для ALU, LSU, MUL/DIV
 - Указание наличия результата
 - Передача информации о PC для отладки

Модуль `biriscv_fetch`

Исходный код модуля представлен в `biriscv_fetch.v`

Назначение Блок выборки инструкций, выполняющий: - Управление процессом загрузки инструкций из кэша - Обработку переходов и изменение потока выполнения - Синхронизацию с конвейером процессора

Ключевые особенности

1. Гибкая архитектура:
 - Поддержка работы с MMU
 - Буферизация запросов и ответов
 - Обработка ошибок доступа

2. Управление потоком:
 - Реакция на переходы
 - Поддержка привилегированных режимов
 - Обработка инвалидации кэша
3. Эффективность:
 - Механизм skid buffer для минимизации простоев
 - Конвейерная обработка запросов
 - Интеграция с системой предсказания

Основные функции

1. Управление кэшем инструкций:
 - Формирование запросов на чтение
 - Обработка подтверждений и ошибок
 - Инвалидация кэша по требованию
2. Обработка переходов:
 - Приоритизация запросов на переход
 - Буферизация адресов переходов
 - Синхронизация с NPC (Next PC) блоком
3. Синхронизация данных:
 - Передача инструкций в декодер
 - Учет предсказаний ветвлений
 - Обработка back-pressure от конвейера

Модуль dcache

Модуль представляет собой кэш данных для RISC-V процессора с поддержкой: - Кэшируемых и некэшируемых запросов - Интерфейса AXI4 для работы с внешней памятью - Поддержки операций инвалидации и writeback - Пайплайнизированных запросов

Параметры

Параметр	Описание
AXI_ID	Идентификатор AXI

Входы и выходы

Входные сигналы процессорного интерфейса

Сигнал	Описание
clk_i	Тактовый сигнал
rst_i	Сигнал сброса
mem_addr_i[31:0]	Адрес памяти
mem_data_wr_i[31:0]	Данные для записи
mem_rd_i	Сигнал чтения
mem_wr_i[3:0]	Сигналы записи (по байтам)
mem_cacheable_i	Флаг кэшируемости доступа
mem_req_tag_i[10:0]	Тег запроса
mem_invalidate_i	Сигнал инвалидации кэша
mem_writeback_i	Сигнал сброса кэша в память
mem_flush_i	Сигнал полного сброса кэша

Входные сигналы AXI интерфейса

Сигнал	Описание
axi_*ready_i	Сигналы готовности AXI
axi_*valid_i	Сигналы валидности данных AXI
axi_*data_i	Входные данные AXI
axi_*resp_i	Коды ответа AXI

Выходные сигналы процессорного интерфейса

Сигнал	Описание
mem_data_rd_o[31:0]	Считанные данные
mem_accept_o	Подтверждение приема запроса
mem_ack_o	Подтверждение выполнения запроса
mem_error_o	Сигнал ошибки
mem_resp_tag_o[10:0]	Тег ответа

Сигнал	Описание
axi_*valid_o	Сигналы валидности AXI
axi_*addr_o	Адреса AXI
axi_*data_o	Выходные данные AXI
axi_*ready_o	Сигналы готовности AXI

Выходные сигналы AXI интерфейса

Подключенные модули

Модуль состоит из 6

Модуль dcache_if_pmem

Назначение

Интерфейсный модуль между кэшем данных и внешней памятью, обеспечивающий: - Буферизацию запросов на чтение/запись - Управление потоком данных - Согласование протоколов

Ключевые особенности

1. Двухканальная буферизация:
 - Отдельные FIFO для запросов и ответов
 - Глубина буферов: 2 элемента каждый
 - Поддержка back-pressure
2. Обработка транзакций:
 - Поддержка 1 невыполненного запроса
 - Фильтрация служебных операций
 - Генерация сигналов подтверждения и ошибок
3. Интерфейсная совместимость:
 - Адаптация к протоколу AXI4/AXI4-Lite
 - Поддержка пакетных операций
 - Выравнивание адресов

Основные функции

1. Управление запросами:
 - Прием запросов от кэша данных
 - Буферизация и маршрутизация
 - Формирование выходных сигналов
2. Обработка ответов:
 - Сопоставление тегов запросов/ответов
 - Передача данных и статусов обратно в кэш
 - Обработка ошибок доступа
3. Служебные операции:
 - Пропуск не критичных запросов
 - Отслеживание состояния выполнения
 - Синхронизация с тактовым сигналом

Модуль `dcache_pmem_mux`

Назначение

Мультиплексор интерфейса памяти кэша данных, обеспечивающий: - Коммутацию между двумя источниками запросов к памяти - Распределение ответов памяти соответствующим источникам - Синхронизацию потоков данных

Ключевые особенности

1. Двухканальная коммутация:
 - Выбор между `inport0` и `inport1` по сигналу `select_i`
 - Полная поддержка всех сигналов интерфейса памяти
2. Синхронизация ответов:
 - Фиксация состояния селектора для корреляции запросов/ответов
 - Задержка сигнала `select` на 1 такт для согласования фаз
3. Прозрачная маршрутизация:
 - Сквозная передача всех параметров запроса
 - Распределение ответов источнику

Основные функции

1. Мультиплексирование запросов:
 - Выбор активного канала по `select_i`
 - Передача `wr/rd` сигналов
 - Маршрутизация адреса и данных записи
2. Демultipлексирование ответов:
 - Направление подтверждений соответствующему источнику
 - Распределение данных чтения и флагов ошибок
3. Управление потоком:
 - Передача сигналов готовности выбранному каналу
 - Синхронизация с тактовым сигналом
 - Обработка сброса

Модуль `dcache_mux`

Назначение

Мультиплексор доступа к кэшируемой и некэшируемой памяти, обеспечивающий: - Маршрутизацию запросов в зависимости от типа памяти - Контроль состояния выполнения операций - Согласование интерфейсов кэшируемого и некэшируемого доступа

Ключевые особенности

1. Два порта:
 - Кэшируемый доступ
 - Прямой доступ к памяти
 - Автоматический выбор на основе `mem_cacheable_i`
2. Управление потоком:
 - Отслеживание невыполненных запросов
 - Блокировка конфликтующих операций
 - Контроль перекрывающихся транзакций
3. Синхронизация:

- Фиксация типа текущего доступа
- Согласование тактовых доменов
- Обработка сброса

Основные функции

1. Маршрутизация запросов:
 - Распределение операций чтения/записи
 - Передача служебных команд
 - Переадресация тегов запросов
2. Обработка ответов:
 - Выбор источника данных для чтения
 - Маршрутизация подтверждений и ошибок
 - Возврат тегов ответов
3. Контроль состояния:
 - Отслеживание активных транзакций
 - Предотвращение конфликтов доступа
 - Индикация активного типа доступа

Модуль dcache_axi

Назначение

Адаптер интерфейса кэша данных к шине AXI, обеспечивающий: - Преобразование протоколов - Управление burst-транзакциями - Буферизацию запросов и ответов

Ключевые особенности

1. Двухнаправленная буферизация:
 - FIFO для исходящих запросов
 - Трекинг невыполненных транзакций
 - Поддержка back-pressure
2. Полная поддержка AXI:
 - Все каналы AXI (чтение, запись, ответы)
 - Поддержка burst-операций
 - Обработка всех сигналов AXI
3. Управление потоком:
 - Контроль количества outstanding-транзакций
 - Счетчик оставшихся burst-операций
 - Маркировка последней операции в burst

Основные функции

1. Интерфейс с кэшем:
 - Прием запросов чтения/записи
 - Возврат данных и статусов
 - Управление готовностью
2. Генерация AXI-транзакций:
 - Формирование заголовков AXI
 - Управление burst-операциями
 - Маршрутизация данных записи
3. Обработка ответов:

- Прием подтверждений
- Обработка ошибок
- Сопоставление ответов с запросами

Модуль `dcache_core`

Назначение

Ядро кэша данных процессора с политикой `write-back`, выполняющее: - Кэширование данных для ускорения доступа - Управление согласованностью данных - Обработку всех типов запросов к памяти

Ключевые особенности

1. Архитектура кэша:
 - 2-канальная `set-associative` организация
 - Размер строки: 32 байта
 - Псевдослучайный алгоритм замещения
2. Политики работы:
 - `Write-back` с выделением строк при чтении/записи
 - Поддержка некешируемых областей памяти
 - Пайплайнинг операций
3. Состояния автомата:
 - 11 состояний (`LOOKUP`, `REFILL`, `EVICT` и др.)
 - Сложная логика переходов между состояниями

Основные функции

1. Обработка запросов:
 - Чтение/запись данных
 - Инвалидация строк
 - Полная очистка кэша (`flush`)
 - Принудительная запись (`writeback`)
2. Управление кэшем:
 - Поиск данных по тегам
 - Заполнение строк при промахх
 - Вытеснение “грязных” строк
 - Обновление метаданных
3. Интерфейс с памятью:
 - Генерация `burst`-транзакций
 - Обработка подтверждений
 - Контроль ошибок

Особенности реализации

- Двойные порты для RAM тегов и данных
- Глубокая конвейеризация операций
- Механизмы предотвращения конфликтов
- Поддержка отладки

Модуль riscv_core

Модуль представляет собой реализацию процессора RISC-V с поддержкой:

1. Конвейерной обработки команд
2. Кэширования инструкций и данных
3. Предсказания переходов
4. Привилегированных режимов
5. MMU

Параметры

Параметр	Описание
SUPPORT_BRANCH_PREDICTION	Включение предсказания переходов
SUPPORT_MULDIV	Поддержка умножения/деления
SUPPORT_SUPER	Поддержка привилегированных режимов
SUPPORT_MMU	Поддержка MMU
SUPPORT_DUAL_ISSUE	Поддержка двухпоточной выдачи
SUPPORT_LOAD_BYPASS	Обход загрузки
SUPPORT_MUL_BYPASS	Обход умножения
SUPPORT_REGFILE_XILINX	Использование регистрового файла Xilinx
EXTRA_DECODE_STAGE	Доп. стадия декодирования
MEM_CACHE_ADDR_MIN	Мин. адрес кэшируемой памяти
MEM_CACHE_ADDR_MAX	Макс. адрес кэшируемой памяти
NUM_BTBT_ENTRIES	Число записей в BTB
NUM_BHT_ENTRIES	Число записей в BHT
RAS_ENABLE	Включение стека возвратов
GSHARE_ENABLE	Включение схемы Gshare
BHT_ENABLE	Включение истории переходов
NUM_RAS_ENTRIES	Глубина стека возвратов

Входы и выходы

Подключенные модули

Модуль состоит из шести подмодулей:

Модуль biriscv_frontend

Описание модуля представлено в документе `biriscv_frontend.md`

Исходный код модуля представлен в `biriscv_frontend.v`

Фронтенд 32-битного RISC-V процессора, отвечающий за: - Предсказание переходов - Выбор инструкций из памяти - Предварительное декодирование инструкций - Формирование конвейера инструкций

Ключевые особенности

- Поддержка двухпоточного исполнения
- Конфигурируемые параметры предсказания переходов

Порт	Описание
Входные сигналы	
clk_i	Тактовый сигнал
rst_i	Сигнал сброса
mem_d_data_rd_i	Данные чтения из памяти данных
mem_d_accept_i	Подтверждение приема запроса данных
mem_d_ack_i	Подтверждение выполнения запроса
mem_d_error_i	Ошибка доступа к данным
mem_d_resp_tag_i	Тег ответа данных
mem_i_accept_i	Подтверждение приема запроса инструкций
mem_i_valid_i	Действительные данные инструкций
mem_i_error_i	Ошибка доступа к инструкциям
mem_i_inst_i	Инструкции из памяти
intr_i	Вход прерывания
reset_vector_i	Вектор сброса
cpu_id_i	Идентификатор CPU
Выходные сигналы	
mem_d_addr_o	Адрес данных
mem_d_data_wr_o	Данные для записи
mem_d_rd_o	Сигнал чтения данных
mem_d_wr_o	Сигналы записи данных
mem_d_cacheable_o	Флаг кэшируемости
mem_d_req_tag_o	Тег запроса данных
mem_d_invalidate_o	Инвализация кэша
mem_d_writeback_o	Запись обратно
mem_d_flush_o	Сброс кэша
mem_i_rd_o	Сигнал чтения инструкций
mem_i_flush_o	Сброс кэша инструкций
mem_i_invalidate_o	Инвализация кэша инструкций
mem_i_pc_o	Адрес инструкции

- Branch Target Buffer
- Branch History Table
- Return Address Stack
- Поддержка MMU
- Раздельные интерфейсы для кэша инструкций и декодера
- Обнаружение ошибок доступа к памяти

Модуль `biriscv_mmu`

Исходный код модуля представлен в `biriscv_mmu.v`

Назначение Блок управления виртуальной памятью для RISC-V процессора, выполняющий: - Конвертацию виртуальных адресов в физические - Контроль прав доступа к памяти - Управление TLB - Обработку страничных исключений

Ключевые особенности

- Поддержка страничной адресации RISC-V
- Двухуровневая таблица страниц (4KB / 4MB)
- Раздельные TLB для инструкций и данных
- Контроль прав доступа:
 - Чтение/запись/исполнение
 - Пользовательский и привилегированный режимы
 - Флаги SUM и MXR для гибкого контроля
- Поддержка аппаратной инвалидации TLB

Основные функции

1. Обработка запросов на выборку инструкций:
 - Проверка прав исполнения
 - Контроль доступа супервизора к пользовательским страницам
2. Обработка запросов к данным:
 - Проверка прав чтения и записи
 - Учет флага MXR - разрешение исполнения при чтении
 - Учет флага SUM - разрешение супервизору доступа к пользовательским страницам
3. Page Table Walker:
 - Аппаратный обход таблиц страниц
 - Поддержка двухуровневой адресации
 - Обработка ошибок
4. Интерфейс с кэшем:
 - Генерация физических адресов
 - Управление кэшируемостью памяти
 - Обработка исключений доступа

Модуль `biriscv_lsu`

Исходный код модуля представлен в `biriscv_lsu.v`

Назначение Блок загрузки и сохранения для процессора, выполняющий: - Обработку операций загрузки и сохранения - Управление доступом к памяти данных - Обработку неполных адресов - Генерацию исключений при ошибках доступа

Ключевые особенности

- Поддержка операций разной ширины (8/16/32 бита)
- Обработка знаковых/беззнаковых загрузок
- Поддержка кэшируемых и некэшируемых областей памяти
- Управление кэшем данных
- Конвейерная обработка запросов

Основные функции

1. Декодирование инструкций:
 - Определение типа операции
 - Вычисление эффективного адреса
 - Проверка выравнивания адреса
2. Формирование запросов к памяти:
 - Генерация сигналов чтения/записи
 - Маскировка байтов для операций разной ширины
 - Управление кэшэм
3. Обработка ответов:
 - Коррекция данных при загрузке
 - Обработка ошибок
 - Формирование результата для блока writeback
4. Управление конвейером:
 - Генерация сигналов остановки
 - Отслеживание невыполненных запросов

Модуль biriscv_csr

Исходный код модуля представлен в `biriscv_csr.v`

Назначение Блок управления Control and Status Registers, отвечающий за: - Обработку операций с CSR-регистрами - Управление привилегированными режимами - Обработку исключений и прерываний - Управление MMU и кэшем - Генерацию сигналов перехода

Ключевые особенности

- Поддержка стандартных CSR RISC-V
- Обработка инструкций: ECALL/EBREAK/ERET, FENCE, WFI
- Управление привилегиями доступа к CSR
- Интеграция с системой прерываний
- Поддержка супервизорного режима

Основные функции

1. Декодирование CSR-инструкций:
 - CSRRW/CSRRS/CSRRC и их immediate-варианты
 - Проверка прав доступа к CSR
2. Управление состоянием процессора:
 - Обработка исключений и прерываний
 - Управление привилегированными режимами
 - Обработка инструкций синхронизации
3. Интеграция с конвейером:

- Генерация сигналов исключений
 - Управление переходами
 - Взаимодействие с MMU
4. Системные операции:
- Сброс процессора
 - Управление таймерами и внешними прерываниями

Модуль `biriscv_multiplier`

Исходный код модуля представлен в `biriscv_multiplier.v`

Назначение Блок умножения, выполняющий: - Целочисленное умножение 32-битных операндов - Обработку знаковых и беззнаковых операций - Конвейерную обработку с настраиваемой глубиной

Ключевые особенности

- Поддержка всех типов умножения RISC-V:
 - MUL
 - MULH
 - MULHU
 - MULHSU
- Конфигурируемая задержка
- Интеграция с конвейером процессора

Основные функции

1. Декодирование инструкций:
 - Определение типа операции умножения
 - Подготовка операндов
2. Выполнение умножения:
 - 32×32-битное умножение с получением 64-битного результата
 - Выбор нужной части результата
3. Конвейерная обработка:
 - Регистры для промежуточного хранения результатов
 - Поддержка сигнала остановки конвейера

Модуль `biriscv_divider`

Исходный код модуля представлен в `biriscv_divider.v`

Назначение Блок деления, выполняющий:

- Деление и вычисление остатка для 32-битных операндов
- Обработку знаковых и беззнаковых операций
- Пошаговое вычисление за несколько тактов

Ключевые особенности

- Поддержка всех операций деления RISC-V:
 - DIV (знаковое деление)

- DIVU (беззнаковое деление)
- REM (знаковый остаток)
- REMU (беззнаковый остаток)
- Последовательный алгоритм деления
- Оптимизация повторных операций с теми же операндами
- Коррекция знака результата

Основные функции

1. Декодирование инструкций:
 - Определение типа операции (DIV/DIVU/REM/REMU)
 - Подготовка операндов
2. Выполнение деления:
 - Последовательное вычитание со сдвигом
 - Формирование частного и остатка
 - Коррекция знака результата
3. Управление процессом:
 - Контроль состояния вычисления
 - Определение завершения операции
 - Формирование выходных данных для блока writeback

Модуль biriscv_issue

Исходный код модуля представлен в biriscv_issue.v

Назначение Блок управления конвейером и диспетчеризации инструкций, выполняющий: - Управление порядком выполнения инструкций - Распределение инструкций по исполнительным блокам - Обработку зависимостей данных - Управление обходными путями - Контроль состояния конвейера

Ключевые особенности

- Поддержка двухканального вызова инструкций
- Управление 5 основными исполнительными блоками:
 - ALU
 - LSU
 - MUL
 - DIV
 - CSR
- Динамическое планирование с учетом зависимостей
- Поддержка прерываний и исключений

Основные функции

1. Декодирование и диспетчеризация:
 - Анализ готовности инструкций
 - Распределение по исполнительным блокам
 - Управление двухканальным выпуском
2. Управление зависимостями:
 - Scoreboarding для отслеживания занятости регистров

- Обходные пути для результатов операций
- Обработка RAW/WAW зависимостей
- 3. Управление конвейером:
 - Обработка остановок и сбросов
 - Управление длительными операциями
 - Координация работы с блоками CSR и LSU
- 4. Обработка переходов:
 - Управление предсказанием переходов
 - Обработка ошибочных предсказаний
 - Обновление информации о ветвлениях

Модуль `biriscv_ехес`

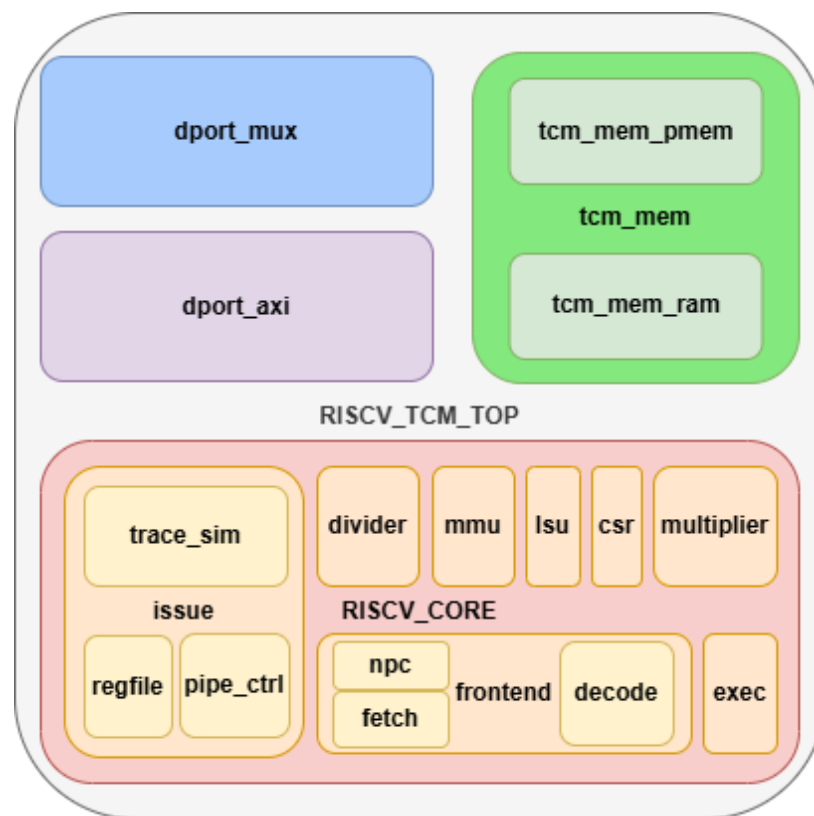
Исходный код модуля представлен в `biriscv_ехес.v`

Назначение Основной блок исполнения команд в процессоре RISC-V, выполняющий:
 - Арифметико-логические операции - Обработку переходов и ветвлений - Вычисление адресов для операций загрузки/сохранения - Формирование результатов для writeback

Ключевые функции

1. Обработка ALU операций:
 - Арифметические (ADD, SUB)
 - Логические (AND, OR, XOR)
 - Сдвиги (SLL, SRL, SRA)
 - Сравнения (SLT, SLTU)
2. Управление потоком команд:
 - Безусловные переходы (JAL, JALR)
 - Условные ветвления (BEQ, BNE, BLT, BGE)
 - Определение признаков call/ret
3. Генерация констант:
 - Формирование immediate-значений
 - Обработка LUI/AUIPC инструкций
4. Интеграция с конвейером:
 - Управление сигналами ветвления
 - Передача результатов в writeback
 - Обработка сигналов остановки

Модуль riscv_tcm_top



Модуль представляет собой суперскалярный процессор RISC-V с двумя АЛУ с Tightly Coupled Memory, объединяющую: суперскалярный процессор с двумя АЛУ - Ядро RISC-V процессора - Локальную память с интерфейсом AXI - Подсистему внешнего доступа к памяти через AXI - Механизмы управления памятью и прерываниями

Тесно связанная память - часть адресуемой памяти, физически расположенной на первом уровне иерархии памяти, рядом с кэш-памятью первого уровня.

Параметры

Параметр	Описание
BOOT_VECTOR	Адрес начальной загрузки
CORE_ID	Идентификатор процессора
TCM_MEM_BASE	Базовый адрес TCM памяти
SUPPORT_*	Флаги поддержки различных функций
MEM_CACHE_ADDR_*	Диапазон адресов кэшируемой памяти
NUM_*_ENTRIES	Размеры таблиц предсказания переходов

Входы и выходы

Системные сигналы

Сигнал	Описание
clk_i	Тактовый сигнал
rst_i	Сигнал сброса системы
rst_cpu_i	Сигнал сброса процессора
intr_i[31:0]	Входы прерываний

Подключенные модули

Модуль состоит из четырех основных подмодулей:

1. riscv_core

Описание модуля представлено в документе `riscv_core.md`

Исходный код модуля представлен в `riscv_core.v`

Ядро процессора, выполняющее инструкции.

Модуль dport_mux

Назначение

Исходный код модуля представлен в `dport_mux.v`

Мультиплексор доступа к памяти для процессора, обеспечивающий: - Маршрутизацию запросов между ТСМ и внешней памятью - Управление потоком команд для разных областей памяти - Согласование интерфейсов доступа

Ключевые особенности

1. Два порта:
 - Доступ к быстрой ТСМ памяти
 - Доступ к основной внешней памяти
 - Автоматическое определение целевой памяти по адресу
2. Управление потоком:
 - Отслеживание невыполненных запросов
 - Блокировка конфликтующих операций
 - Контроль перекрывающихся транзакций
3. Поддержка всех операций:
 - Чтение/запись данных
 - Управляющие команды
 - Учет кэшируемости областей

Основные функции

1. Маршрутизация запросов:
 - Определение целевой памяти по адресу
 - Передача всех параметров запроса
 - Распределение управляющих сигналов
2. Обработка ответов:
 - Выбор источника данных для чтения
 - Маршрутизация подтверждений и ошибок
 - Возврат тегов ответ

Модуль tcm_mem

Исходный код модуля представлен в `tcm_mem.v`

Назначение

Двухпортовая Tightly Coupled Memory для процессора, обеспечивающая: - Низколатентный доступ к инструкциям и данным - Одновременный доступ процессора и внешних устройств через AXI - Управление памятью с возможностью отладки

Ключевые особенности

1. Двухпортовая архитектура:
 - Порт 0: Доступ только для чтения инструкций (64-бит)
 - Порт 1: Доступ для данных (32-бит) и внешних устройств
2. Интерфейсы:
 - Процессорный интерфейс для инструкций и данных
 - Полнофункциональный AXI-интерфейс для внешнего доступа
 - Встроенный преобразователь AXI->PMEM
3. Особенности реализации:
 - Разделение 64-битных слов на верхнюю/нижнюю части
 - Приоритет внешнего доступа над доступом данных процессора
 - Поддержка функций отладки в Verilator

Основные функции

1. Доступ к инструкциям:
 - Фиксированная задержка в 1 такт
 - 64-битная шина данных
 - Бесконфликтный параллельный доступ
2. Доступ к данным:
 - Поддержка операций чтения/записи
 - Байтовая адресация (`wr_i[3:0]`)
 - Конфликтное управление с внешним доступом
3. Внешний доступ через AXI:
 - Поддержка burst-транзакций
 - Полная обработка всех каналов AXI
 - Преобразование в простой интерфейс PMEM
4. Отладочные функции:
 - Функции `read/write` для доступа к памяти из тестов
 - Побайтовый доступ к памяти

Модуль `dport_axi`

Исходный код модуля представлен в `dport_axi.v`

Назначение

Адаптер интерфейса кэша данных к шине AXI, обеспечивающий: - Преобразование протоколов между `dcache_if` и AXI4/AXI4-Lite - Буферизацию транзакций - Управление потоком данных

Ключевые особенности

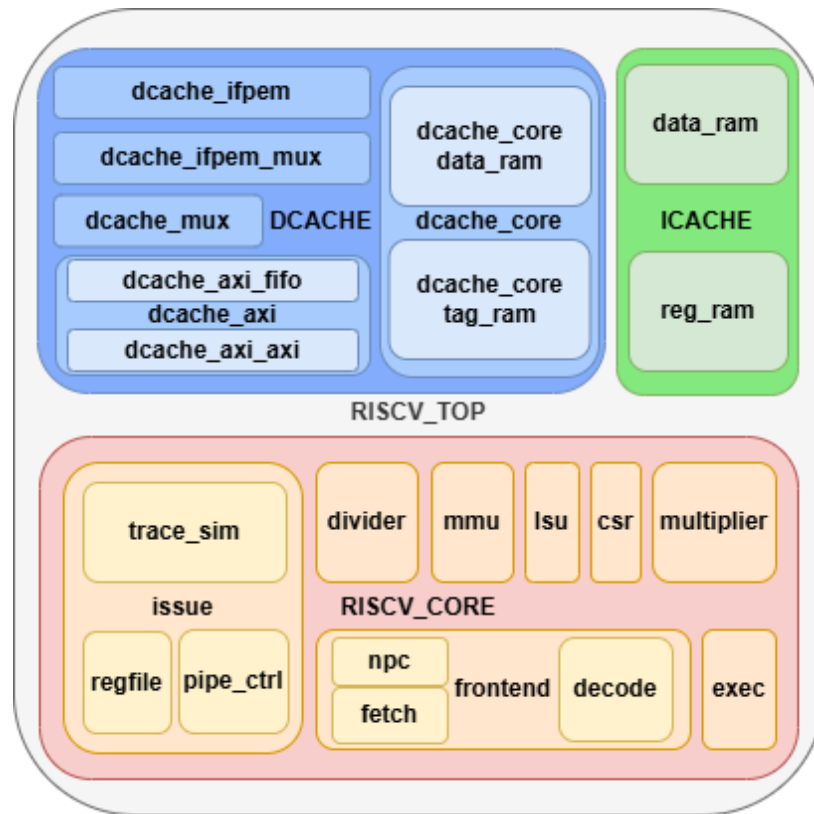
1. Двухнаправленная буферизация:
 - FIFO для запросов

- FIFO для тегов ответов
- Поддержка back-pressure
- 2. Управление транзакциями:
 - Поддержка 1 невыполненного запроса
 - Раздельная обработка чтения/записи
 - Контроль подтверждений операций
- 3. Полная поддержка AXI:
 - Все каналы AXI (чтение, запись, ответы)
 - Обработка ошибок
 - Поддержка AXI-Lite

Основные функции

1. Обработка запросов:
 - Маршрутизация операций чтения/записи
 - Буферизация адресов и данных
 - Управление сигналами готовности
2. Генерация AXI-транзакций:
 - Формирование заголовков AXI
 - Синхронизация каналов AW/W
 - Обработка подтверждений
3. Обработка ответов:
 - Сопоставление тегов запросов/ответов
 - Передача данных чтения
 - Фильтрация ошибок

Модуль riscv_top



Модуль `riscv_top` представляет собой суперскалярный процессор RISC-V с двумя АЛУ, объединяющей ядро процессора, кэш инструкций и кэш данных с интерфейсом AXI. Этот модуль предназначен для реализации RISC-V совместимого процессора с поддержкой различных функций.

Параметры

Порты

Подключенные модули

Модуль состоит из трёх основных подмодулей:

1. `riscv_core`

Описание модуля представлено в документе `risc_core.md`

Исходный код модуля представлен в `riscv_core.v`

Ядро процессора, выполняющее инструкции. Обменивается данными с кэшом через унифицированный интерфейс:

- Интерфейсы:
 - `mem_d_*` — чтение/запись данных
 - `mem_i_*` — запросы инструкций
- Поддерживаемые функции:
 - Предсказание переходов:
 - * BTB (Branch Target Buffer)
 - * BHT (Branch History Table)

Параметр	Описание
CORE_ID	Идентификатор ядра
ICACHE_AXI_ID	AXI ID для кэша инструкций
DCACHE_AXI_ID	AXI ID для кэша данных
SUPPORT_BRANCH_PREDICTION	Включение предсказания переходов
SUPPORT_MULDIV	Поддержка операций умножения/деления
SUPPORT_SUPER	Поддержка привилегированных режимов
SUPPORT_MMU	Поддержка MMU
SUPPORT_DUAL_ISSUE	Поддержка двухпоточной выдачи инструкций
SUPPORT_LOAD_BYPASS	Поддержка обхода (bypass) загрузки
SUPPORT_MUL_BYPASS	Поддержка обхода (bypass) умножения
SUPPORT_REGFILE_XILINX	Использование специализированного регистр-файла
EXTRA_DECODE_STAGE	Добавление дополнительной стадии декодирования
MEM_CACHE_ADDR_MIN	Минимальный адрес кэшируемой памяти
MEM_CACHE_ADDR_MAX	Максимальный адрес кэшируемой памяти
NUM_BTB_ENTRIES	Количество записей в BTB (буфере целевых адресов)
NUM_BHT_ENTRIES	Количество записей в BHT (истории переходов)
NUM_RAS_ENTRIES	Количество записей в RAS (стеке возвратов)
RAS_ENABLE	Включение стека возвратов (RAS)
GSHARE_ENABLE	Включение схемы предсказания Gshare
BHT_ENABLE	Включение истории переходов (BHT)

Порты	Описание
Входы	
clk_i	Системный тактовый сигнал
rst_i	Системный сигнал сброса
axi_i_*	AXI-сигналы от кэш-интерфейса инструкций
axi_d_*	AXI-сигналы от кэш-интерфейса данных
intr_i	Вход внешнего прерывания
reset_vector_i	Начальный адрес вектора сброса
Выходы	
axi_i_*	AXI-сигналы на кэш-интерфейса инструкций
axi_d_*	AXI-сигналы на кэш-интерфейса данных

* RAS (Return Address Stack)

- Параллельная выдача инструкций
- Обход (bypass) для ускорения выполнения

2. icache

Исходный код модуля представлен в `icache.v`

Назначение Кэш-память инструкций, обеспечивающая: - Быстрый доступ к часто используемым инструкциям - Уменьшение задержек при выборке команд - Снижение нагрузки на системную шину

Ключевые особенности

1. Архитектура кэша:
 - 2-канальная set-associative организация
 - Размер строки кэша: 32 байта
 - Псевдослучайный алгоритм замещения
2. Интерфейсы:
 - Внутренний интерфейс для процессора
 - AXI4-Lite интерфейс для доступа к памяти
 - Поддержка инвалидации и сброса кэша
3. Производительность:
 - Однотактный доступ при попадании в кэш
 - Пайплайнинг операций
 - Буферизация запросов

Основные функции

1. Управление кэшем:
 - Поиск данных по тегам
 - Заполнение строк при промахе
 - Инвалидация отдельных строк или всего кэша
2. Обработка запросов:
 - Прием адресов от блока выборки
 - Возврат инструкций или флагов ошибок
 - Управление потоком команд при промахах
3. Работа с памятью:
 - Генерация burst-транзакций AXI
 - Обработка ответов от памяти
 - Контроль ошибок доступа
4. Состояния работы:
 - Обычный режим
 - Заполнение кэша
 - Повторный поиск
 - Сброс кэша

3. dcache

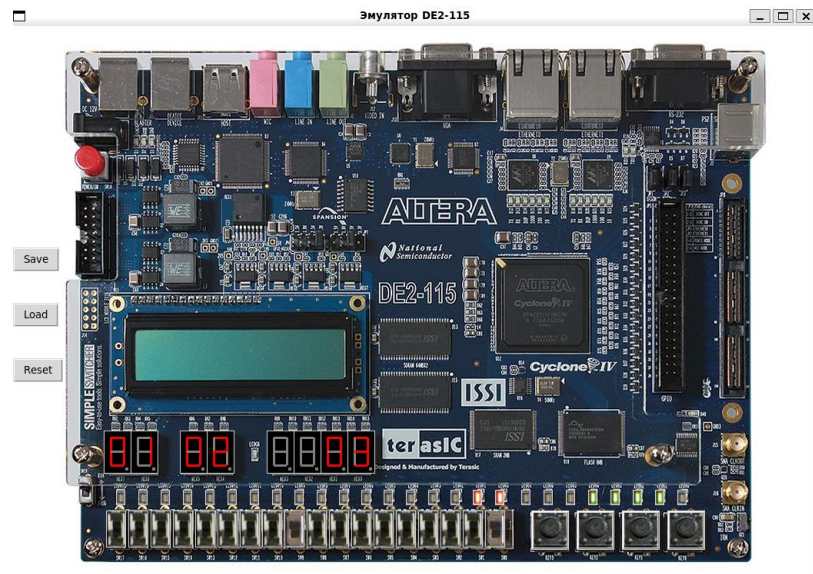
Описание модуля представлено в `documentedcache.md`

Исходный код модуля представлен в `dcache.v`

Кэш данных, подключенный к axi_d_*.

- Особенности:
 - Поддержка кэшируемого и некэшируемого доступа
 - Операции:
 - * Инвалидация
 - * Сброс
 - * Writeback
 - Ускоренный доступ к памяти
- Ключевые интерфейсы:
 - mem_d_* — подключение к ядру
 - axi_d_* — AXI-канал данных

Виртуальная плата DE2-115



Описание проекта

Эмулятор платы DE2-115 с полным функционалом, включая: - Графический интерфейс пользователя - Сетевое взаимодействие с Verilog-симуляцией - 18 переключателей - 18 красных светодиодов - 8 зеленых светодиодов - 8 семисегментных индикаторов - 4 кнопки - Сохранение и загрузка состояний

ОСНОВНЫЕ ВОЗМОЖНОСТИ

- Точная эмуляция компонентов платы DE2-115
- Интерактивное взаимодействие с Verilog-симуляцией
- Адаптивный интерфейс с масштабированием
- Гибкая система конфигурации
- Сохранение положений переключателей и кнопок между сеансами
- Сброс состояния к значениям по умолчанию

Требования

Для графического клиента

- Python 3.6+
- Библиотеки:
 - tkinter
 - Pillow (PIL)
 - json
 - configparser

Для сервера

- Verilator 4.0+
- Компилятор с поддержкой C++17
- Библиотеки:
 - libjsoncpp

– pthread

Установка

1. Клонировать репозиторий:

```
[] git clone https://git.miem.hse.ru/1963/virtual_fpga_boards.git cd virtual_fpga_boards
```

2. Установите зависимости Python:

```
[] pip install pillow
```

3. Соберите сервер:

```
[] make build
```

Использование

Возможен одновременный запуск через Makefile или запуск графического клиента и сервера по отдельности. Во втором случае при запуске графического клиента можно ввести параметры расширения окна.

Запуск через Makefile

```
[] make run
```

Запуск графического клиента

```
[] python app.py [--width ШИРИНА] [--height ВЫСОТА] [--config ФАЙЛ]
```

Параметры:

–width: Ширина окна в пикселях (переопределяет конфиг)

–height: Высота окна в пикселях (переопределяет конфиг)

–config: Путь к файлу конфигурации (по умолчанию: config.ini)

Файл конфигурации

Отредактируйте config.ini для настройки размеров окна:

```
[] [DEFAULT] width = 800 height = 500
```

Запуск сервера

```
[] ./server
```

Функционал графического интерфейса

- Переключатели: 18 переключателей, которые можно включать и выключать. Их состояние сохраняется между сеансами.
- Кнопки: 4 кнопки, которые остаются “нажатыми” после клика и возвращаются в исходное состояние при повторном клике.
- Красные светодиоды: 18 светодиодов, отображающих состояние.
- Зеленые светодиоды: 8 светодиодов, отображающих режимы работы.

- Семисегментные индикаторы: 8 индикаторов, отображающих значения.
- Кнопка “Save”: Сохраняет текущее состояние переключателей и кнопок в файл state.pkl.
- Кнопка “Load”: Загружает сохраненное состояние из файла state.pkl.
- Кнопка “Reset”: Сбрасывает состояние переключателей и кнопок к значениям по умолчанию.

Makefile

Доступные команды:

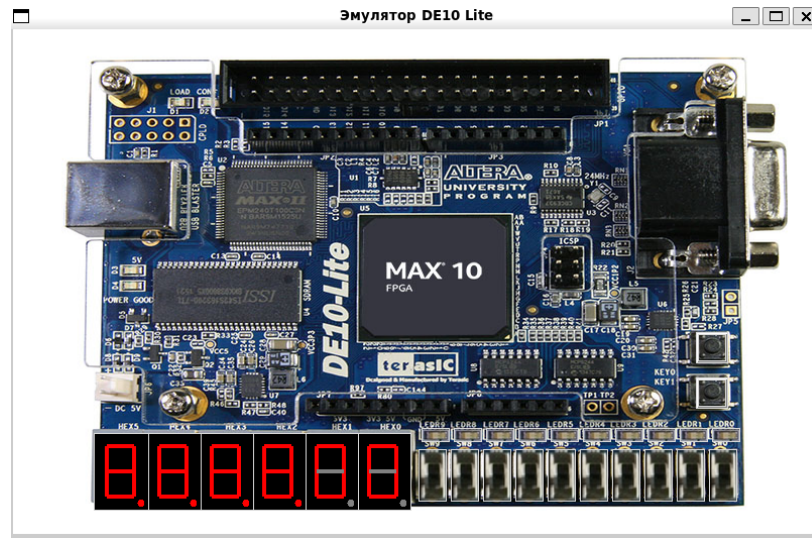
Команда	Описание
make	Показывает справку по доступным командам
make build	Собирает проект (верилирует Verilog и компилирует C++ код)
make run	Собирает и запускает сервер
make clean	Удаляет все сгенерированные файлы сборки
make check_python	Проверяет версию Python (должна быть 3.6+)

Таблица сигналов

На виртуальной плате DE2-115 загружено 16-битное АЛУ, которое поддерживает арифметические и логические операции. Ниже приведено описание сигналов, подключенных к элементам платы.

Сигнал	Описание	Подключ
alu_result[15:0]	Результат работы АЛУ	LEDR0-LEDR15,
alu_cout	Перенос (Carry Out)	LEDG
a_reg[15:0]	Регистр операнда A	SW7-SW0 и KEY0 (зап
b_reg[15:0]	Регистр операнда B	SW7-SW0 и KEY0 (зап
sel[3:0]	Код операции АЛУ	SW14-SV
mode	Режим работы АЛУ (арифметический/логический)	SW15
carry_in	Входной перенос (Carry In)	SW8

Виртуальная плата DE10-Lite



Описание проекта

Эмулятор платы DE10-Lite с полным функционалом, включая: - Графический интерфейс пользователя - Сетевое взаимодействие с Verilog-симуляцией - 10 переключателей - 10 светодиодов - 6 семисегментных индикаторов - 2 кнопки - Сохранение и загрузка состояний

Основные возможности

- Точная эмуляция компонентов платы DE10-Lite
- Интерактивное взаимодействие с Verilog-симуляцией
- Адаптивный интерфейс с масштабированием
- Гибкая система конфигурации
- Сохранение положений переключателей и кнопок между сеансами
- Сброс состояния к значениям по умолчанию

Требования

Для графического клиента

- Python 3.6+
- Библиотеки:
 - tkinter
 - Pillow (PIL)
 - json
 - configparser

Для Verilog сервера

- Verilator 4.0+
- Компилятор с поддержкой C++17
- Библиотеки:
 - libjsoncpp
 - pthread

Установка

1. Клонировать репозиторий:

```
|| git clone https://git.miem.hse.ru/1963/virtual_fpga_boards.git cd virtual_fpga_boards
```

2. Установите зависимости Python:

```
|| pip install pillow
```

3. Соберите сервер:

```
|| make build
```

Использование

Возможен одновременный запуск через Makefile или запуск графического клиента и сервера по отдельности. Во втором случае при запуске графического клиента можно ввести параметры расширения окна.

Запуск через Makefile

```
|| make run
```

Запуск графического клиента

```
|| python app.py [--width ШИРИНА] [--height ВЫСОТА] [--config ФАЙЛ]
```

Параметры:

- width: Ширина окна в пикселях (переопределяет конфиг)
- height: Высота окна в пикселях (переопределяет конфиг)
- config: Путь к файлу конфигурации (по умолчанию: config.ini)

Файл конфигурации

Отредактируйте config.ini для настройки размеров окна:

```
|| [DEFAULT] width = 800 height = 500
```

Запуск сервера

```
|| ./server
```

Функционал графического интерфейса

- Переключатели: 10 переключателей, которые можно включать и выключать. Их состояние сохраняется между сеансами.
- Кнопки: 2 кнопки, которые остаются “нажатыми” после клика и возвращаются в исходное состояние при повторном клике.
- Светодиоды: 10 светодиодов, отображающих состояние.
- Семисегментные индикаторы: 6 индикаторов, отображающих значения.
- Кнопка “Save”: Сохраняет текущее состояние переключателей и кнопок в файл state.pkl.
- Кнопка “Load”: Загружает сохраненное состояние из файла state.pkl.

- Кнопка “Reset”: Сбрасывает состояние переключателей и кнопок к значениям по умолчанию.

Makefile

Доступные команды:

Команда	Описание
make	Показывает справку по доступным командам
make build	Собирает проект (верилирует Verilog и компилирует C++ код)
make run	Собирает и запускает сервер
make clean	Удаляет все сгенерированные файлы сборки
make check_python	Проверяет версию Python (должна быть 3.6+)

Таблица сигналов

На виртуальной плате DE10-Lite загружено 8-битное АЛУ, которое поддерживает арифметические и логические операции. Ниже приведено описание сигналов, подключенных к элементам платы.

Сигнал	Описание	Подключение
alu_result[7:0]	Результат работы АЛУ	LEDR0-LEDR7, HEX0-H
alu_cout	Перенос (Carry Out)	LEDR8
a_reg[7:0]	Регистр операнда A	SW[3:0] и KEY[0] (запи
b_reg[7:0]	Регистр операнда B	SW[3:0] и KEY[0] (запи
sel[3:0]	Код операции АЛУ	SW[7:4]
mode	Режим работы АЛУ (арифметический/логический)	KEY[1]